1. What is Machine Learning?

Machine learning is the study of algorithms that improve at a task by learning patterns from data instead of following hand-written rules. A model is trained on examples and then used to make predictions on new, unseen inputs.

There are three main paradigms. In supervised learning every training example has a label, for example an email marked as spam or not spam, and the model learns a mapping from inputs to labels. Regression predicts a continuous number such as a house price, while classification predicts a category.

In unsupervised learning the data has no labels. The goal is to discover structure, for example grouping customers into segments with clustering, or compressing data with dimensionality reduction.

In reinforcement learning an agent interacts with an environment and receives rewards or penalties. It learns a policy that maximises the total reward over time, which is how game-playing agents and some robot controllers are trained.

A typical workflow is: collect and clean data, choose features, split the data, train a model, evaluate it on held-out data, and only then deploy it.

2. Linear Regression

Linear regression models the target as a weighted sum of the input features plus a bias term: y_hat = w1*x1 + w2*x2 + ... + b. Training means finding the weights that make predictions as close as possible to the true values.

The usual cost function is the mean squared error (MSE): the average of the squared differences between predictions and true values. Squaring penalises large errors much more than small ones and makes the cost smooth and convex.

There are two ways to find the best weights. The normal equation solves for them directly in closed form, but it requires inverting a matrix, which becomes slow when there are many features. Gradient descent instead improves the weights step by step and scales to large datasets.

Before training it is good practice to scale features to similar ranges, because features with very different scales make gradient descent zig-zag and converge slowly.

3. Gradient Descent

Gradient descent is an iterative optimisation algorithm. At each step it computes the gradient of the cost function with respect to the parameters and moves the parameters a small step in the opposite direction, which is the direction of steepest decrease.

The learning rate controls the size of each step. If it is too small, training needs a very large number of iterations. If it is too large, the updates overshoot the minimum and the cost can oscillate or even diverge.

Batch gradient descent uses the whole training set to compute each update, which is accurate but slow for big datasets. Stochastic gradient descent (SGD) updates the parameters after every single example; it is fast and noisy, and the noise can help it escape shallow local minima. Mini-batch gradient descent is the common compromise: it uses small batches of, for example, 32 or 64 examples.

For a convex cost such as the MSE of linear regression, gradient descent with a suitable learning rate converges to the global minimum. Training is usually stopped when the cost stops improving noticeably.

4. Logistic Regression and Classification

Despite its name, logistic regression is a classification algorithm. It computes a weighted sum of the features and passes it through the sigmoid function, sigma(z) = 1 / (1 + e^-z), which squashes any real number into the range 0 to 1 so the output can be read as a probability.

To turn the probability into a class, a threshold is applied, usually 0.5. The set of points where the predicted probability equals the threshold is called the decision boundary; for plain logistic regression this boundary is linear.

Logistic regression is trained by minimising the cross-entropy loss, also called log loss. It heavily penalises confident predictions that turn out to be wrong. MSE is not used here because combined with the sigmoid it produces a non-convex cost.

For more than two classes, the softmax function generalises the sigmoid and outputs a probability for every class.

5. Overfitting, Underfitting and Regularisation

A model overfits when it learns the noise in the training data instead of the underlying pattern: training error is very low but error on new data is high. A model underfits when it is too simple to capture the pattern, so both training and test error are high.

This is described by the bias-variance tradeoff. High bias means strong, wrong assumptions and leads to underfitting. High variance means the model is too sensitive to the particular training set and leads to overfitting. Good models balance the two.

Regularisation reduces overfitting by adding a penalty on large weights to the cost function. L2 regularisation (ridge) adds the sum of squared weights and shrinks all weights smoothly. L1 regularisation (lasso) adds the sum of absolute weights and tends to drive some weights exactly to zero, which performs automatic feature selection.

Other remedies for overfitting are collecting more training data, using a simpler model, early stopping, and dropout in neural networks.

6. Model Evaluation

Data is normally split into three parts. The training set is used to fit the model, the validation set is used to tune hyperparameters and choose between models, and the test set is used only once at the end to estimate performance on unseen data.

When data is scarce, k-fold cross-validation gives a more reliable estimate: the data is divided into k folds, the model is trained k times, each time holding out a different fold for evaluation, and the k scores are averaged.

Accuracy is the fraction of correct predictions, but it can be misleading on imbalanced datasets. If only 1% of transactions are fraudulent, a model that always predicts 'not fraud' is 99% accurate and completely useless.

The confusion matrix counts true positives, false positives, true negatives and false negatives. Precision is TP / (TP + FP): of the items predicted positive, how many really are. Recall is TP / (TP + FN): of the real positives, how many were found. The F1 score is the harmonic mean of precision and recall.

7. Decision Trees and Ensembles

A decision tree makes predictions by asking a sequence of yes/no questions about the features. Each internal node tests one feature, each branch is an answer, and each leaf holds a prediction. Trees are easy to interpret but overfit easily if they grow too deep.

To choose a split, the tree measures impurity. Entropy measures disorder in the labels, and information gain is the reduction in entropy produced by a split. Gini impurity is a cheaper alternative that usually gives similar trees. Depth limits and pruning control overfitting.

Ensembles combine many models. A random forest trains many trees on bootstrap samples of the data (bagging) and also considers only a random subset of features at each split, then averages their votes. This reduces variance compared with a single tree.

Boosting trains models sequentially, where each new model focuses on the examples the previous ones got wrong. Gradient boosting is a popular and very accurate family of boosting methods for tabular data.

8. k-Nearest Neighbours and k-Means

k-nearest neighbours (k-NN) is a supervised method. To classify a new point, it finds the k closest training points and takes a majority vote of their labels. It has no real training phase, which is why it is called a lazy learner, but prediction is slow because it compares against all stored points.

The choice of k matters: a very small k is sensitive to noise, while a very large k blurs class boundaries. Euclidean distance is the usual metric, and features must be scaled so that no single feature dominates the distance.

k-means is an unsupervised clustering algorithm. It places k centroids, assigns every point to its nearest centroid, moves each centroid to the mean of its assigned points, and repeats until the assignments stop changing.

k-means needs the number of clusters in advance. The elbow method plots the within-cluster sum of squares against k and picks the k where the curve bends. Because results depend on the starting centroids, the k-means++ initialisation spreads the initial centroids apart and gives more reliable clusters.

9. Neural Networks

A neural network is built from layers of simple units called neurons. Each neuron computes a weighted sum of its inputs plus a bias and applies a non-linear activation function. A single neuron with a step activation is the perceptron, which can only separate classes with a straight line.

Common activation functions are the sigmoid, tanh and ReLU. ReLU outputs max(0, x); it is cheap to compute and is the default choice for hidden layers because it reduces the vanishing gradient problem.

Networks are trained with backpropagation: after a forward pass computes the loss, the chain rule is applied backwards through the layers to compute the gradient of the loss with respect to every weight, and an optimiser such as gradient descent updates the weights.

In very deep networks with sigmoid or tanh activations, gradients can shrink as they flow backwards until early layers barely learn; this is the vanishing gradient problem. Dropout is a regularisation technique that randomly switches off a fraction of neurons during training so the network cannot rely on any single neuron.

10. Dimensionality Reduction and PCA

High-dimensional data causes problems known as the curse of dimensionality: points become sparse, distances become less meaningful, and models need far more data to generalise.

Principal component analysis (PCA) reduces dimensionality by projecting the data onto a smaller number of new axes called principal components. The components are the eigenvectors of the data's covariance matrix, ordered by their eigenvalues, so the first component captures the direction of greatest variance.

The explained variance ratio tells how much of the total variance each component keeps. A common rule is to keep enough components to explain around 95% of the variance.

Features should be standardised to zero mean and unit variance before PCA; otherwise features measured on large scales dominate the components. PCA is linear and unsupervised: it ignores the labels.